

Lezione: Introduzione a TypeScript

Cos'è TypeScript?

- Superset di JavaScript sviluppato da Microsoft
- Aggiunge tipizzazione statica e altre funzionalità
- Compilato in JavaScript standard

Perché usare TypeScript?

- Maggiore sicurezza grazie alla tipizzazione
- Migliore supporto agli editor (autocomplete, refactoring)
- Facilita la manutenzione di progetti grandi

Installazione

```
npm install -g typescript
```

Compilare un file `.ts` :

```
tsc nomefile.ts
```

Primo esempio

```
let messaggio: string = "Ciao, TypeScript!";  
console.log(messaggio);
```

Come compilare e provare TypeScript

1. Scrivi il codice in un file `.ts`

Ad esempio, crea un file chiamato `esempio.ts`.

2. Compila il file TypeScript in JavaScript

Da terminale, esegui:

```
tsc esempio.ts
```

Verrà generato un file `esempio.js`.

3. Esegui il file JavaScript con Node.js

```
node esempio.js
```

4. Prova online

Esercizio: Somma di due numeri

1. Crea un file `somma.ts` con il seguente contenuto:

```
function somma(a: number, b: number): number {  
    return a + b;  
}  
  
console.log(somma(5, 7));
```

2. Compila il file TypeScript:

```
tsc somma.ts
```

Verrà generato un file `somma.js` nella stessa cartella.

3. Visualizza il file JavaScript generato per vedere come TypeScript lo ha

Tipi di base

- `string`
- `number`
- `boolean`
- `any`
- `void`
- `null` e `undefined`

Sintassi di base

Dichiarazione di variabili

- `let` e `const` sono usati per dichiarare variabili.
- `let` permette di riassegnare il valore, `const` no.

```
let contatore: number = 5;  
const nomeCorso: string = "TypeScript Base";
```

Tipizzazione delle variabili

- Si specifica il tipo dopo i due punti :

```
let attivo: boolean = true;  
let prezzo: number = 19.99;  
let descrizione: string = "Corso avanzato";
```

Dichiarazione di funzioni

- Si specificano i tipi dei parametri e del valore di ritorno

```
function saluta(nome: string): string {  
    return "Ciao, " + nome;  
}
```

- Parametri opzionali con `?:`:

```
function stampaMessaggio(msg?: string): void {  
    console.log(msg || "Nessun messaggio");  
}
```

- Parametri di default:

```
function incrementa(x: number, incremento: number = 1): number {  
    return x + incremento;  
}
```

Tipi di ritorno `void` e `any`

- `void` indica che la funzione non restituisce nulla
- `any` permette qualsiasi tipo (da usare con cautela)

```
function logMessaggio(msg: string): void {  
    console.log(msg);  
}
```

```
let valore: any = 10;  
valore = "test";
```

Esempi

```
let nome: string = "Mario";  
let eta: number = 30;  
let isStudente: boolean = true;  
let valoreQualsiasi: any = 42;
```

Array e Tuple

```
let numeri: number[] = [1, 2, 3];  
let tuple: [string, number] = ["età", 25];
```

Enum

```
enum Colore {  
    Rosso,  
    Verde,  
    Blu,  
}  
let c: Colore = Colore.Verde;
```

Funzioni

```
function somma(a: number, b: number): number {  
    return a + b;  
}
```


Parametri opzionali e di default

```
function saluta(nome: string = "ospite"): void {  
    console.log("Ciao, " + nome);  
}
```

Oggetti e Interfacce

```
interface Persona {  
    nome: string;  
    eta: number;  
}
```

```
let persona: Persona = { nome: "Anna", eta: 28 };
```

Classi

```
class Animale {  
  nome: string;  
  constructor(nome: string) {  
    this.nome = nome;  
  }  
  parla(): void {  
    console.log(this.nome + " fa un verso.");  
  }  
}  
  
let cane = new Animale("Fido");  
cane.parla();
```

Ereditarietà

```
class Cane extends Animale {  
    parla(): void {  
        console.log(this.nome + " abbaia.");  
    }  
}  
  
let dog = new Cane("Rex");  
dog.parla();
```

Tipi Unione e Type Alias

```
/**
 * Alias per il tipo ID, che può essere un numero o una stringa.
 *
 * Questo tipo viene utilizzato per rappresentare identificatori che possono assumere
 * sia valori numerici che stringhe, ad esempio per gestire ID provenienti da fonti diverse
 * (database, API esterne, ecc.).
 *
 * Esempi d'uso:
 * - let userId: ID = 123;
 * - let sessionId: ID = "abc-123";
 */
type ID = number | string;
let userId: ID = 123;
userId = "abc";
```

Generics

```
function identity<T>(arg: T): T {  
    return arg;  
}  
  
let output = identity<string>("ciao");
```

Moduli

esempio:

```
// file: saluti.ts
export function saluta(nome: string) {
  return `Ciao, ${nome}`;
}

// file: main.ts
import { saluta } from "./saluti";
console.log(saluta("Luca"));
```

Ambienti di sviluppo consigliati

- Visual Studio Code
- Plugin TypeScript

Esercizio 1

Definisci un'interfaccia `Libro` con titolo, autore e anno. Crea una funzione che accetta un array di libri e stampa i titoli.

Soluzione Esercizio 1

```
interface Libro {  
  titolo: string;  
  autore: string;  
  anno: number;  
}  
  
function stampaTitoli(libri: Libro[]): void {  
  libri.forEach((libro) => {  
    console.log(libro.titolo);  
  });  
}  
  
// Esempio di utilizzo:  
const libri: Libro[] = [  
  { titolo: "Il Signore degli Anelli", autore: "J.R.R. Tolkien", anno: 1954 },  
  { titolo: "1984", autore: "George Orwell", anno: 1949 },  
];  
  
stampaTitoli(libri);
```

Esercizio 2

Crea una classe `Studente` con proprietà nome e matricola, e un metodo che stampa una presentazione.

Soluzione Esercizio 2

```
class Studente {  
  nome: string;  
  matricola: number;  
  
  constructor(nome: string, matricola: number) {  
    this.nome = nome;  
    this.matricola = matricola;  
  }  
  
  presentazione(): void {  
    console.log(  
      `Ciao, sono ${this.nome} e la mia matricola è ${this.matricola}.`,  
    );  
  }  
}  
  
// Esempio di utilizzo:  
const studente = new Studente("Giulia", 12345);  
studente.presentazione();
```

Esercizio 3

Scrivi una funzione generica che accetta un array di qualsiasi tipo e restituisce il primo elemento.

Soluzione Esercizio 3

```
function primoElemento<T>(array: T[]): T | undefined {  
    return array[0];  
}
```

// Esempio di utilizzo:

```
console.log(primoElemento([10, 20, 30])); // 10  
console.log(primoElemento(["a", "b", "c"])); // "a"
```

Risorse utili

- [TypeScript Handbook](#)
- [Playground TypeScript](#)